DBMS Lab Assignment 7

Objective:
The objective of this lab is to explore. These functions allow performing calculations across a set of rows that are related to the current row, providing deeper insights into data trends.

The lab will cover:

1. **Aggregate Window Functions**: `SUM()`, `AVG()`, `MAX()`, `MIN()`, etc.
2. **Ranking Window Functions**: `RANK()`, `DENSE_RANK()`, and `ROW_NUMBER()`

We'll use a familiar dataset from previous labs involving **instructors** and **students**.

Introduction to Window Functions

Window functions perform calculations on a set of table rows defined by a **window**, returning a value for each row within the result set. Unlike aggregate functions, window functions do not group rows together into a single output.

Syntax of a Window Function

```
SELECT column1,

    window_function() OVER (PARTITION BY column2 ORDER BY column3) AS
alias_name

FROM table_name;
```

- **PARTITION BY**: Splits the result set into groups (optional).
- **ORDER BY**: Specifies the order of rows within a partition.
- **OVER**: Defines the window for the function.

Dataset Used

Instructor Table:

ID	Name	Dept_Name	Salary
1	Alice	ComputerSci	100000
2	Bob	Math	90000
3	Charlie	Physics	85000
4	David	ComputerSci	95000
5	Eve	Math	70000

Student Table:

Student_ID	Name	Dept_Name	Marks
1	Sarah	ComputerSci	85
2	Mike	Math	90
3	John	Physics	75
4	Alice	Math	92
5	Emma	ComputerSci	88

Section 1: Aggregate Window Functions

Aggregate window functions allow calculations such as **SUM, AVG, MIN, MAX, COUNT** across a window or partition.

Example 1: Cumulative Salary by Department

```
SELECT name, dept_name, salary,
    SUM(salary) OVER (PARTITION BY dept_name ORDER BY salary) AS
cumulative_salary
FROM instructor;
```

Explanation:

- This query calculates the cumulative salary for each department.
- The **PARTITION BY dept_name** ensures the cumulative sum resets for each department.

Practice Questions: Aggregate Window Functions

Question:
Write a query to calculate the **average marks** for each department using a window function.
Solution:

```
SELECT name, dept_name, marks,
    AVG(marks) OVER (PARTITION BY dept_name) AS avg_marks
FROM student;
```

Question:
Write a query to find the **maximum salary** within each department and list it alongside each instructor's details.
Solution:

```
SELECT name, dept_name, salary,
    MAX(salary) OVER (PARTITION BY dept_name) AS max_salary
FROM instructor;
```

Section 2: Ranking Window Functions

Ranking window functions are used to assign ranks or row numbers to rows in a dataset. Common functions include:

- **RANK()**: Assigns a rank to each row with ties, causing gaps in ranking.
- **DENSE_RANK()**: Assigns a rank without gaps.
- **ROW_NUMBER()**: Assigns a unique row number to each row.

Example 2: RANK() Function for Instructors by Salary

```
SELECT name, dept_name, salary,
    RANK() OVER (PARTITION BY dept_name ORDER BY salary DESC) AS rank
```

```
FROM instructor;
```

Explanation:

- This query ranks instructors within each department based on salary (highest salary gets rank 1).
- Tied salaries will receive the same rank, leaving gaps in the ranking.

Practice Questions: Ranking Window Functions

Question:
Write a query using **DENSE_RANK()** to assign ranks to students based on their marks within each department.
Solution:

```
SELECT name, dept_name, marks,
    DENSE_RANK() OVER (PARTITION BY dept_name ORDER BY marks DESC) AS rank
FROM student;
```

Question:
Write a query using **ROW_NUMBER()** to assign unique row numbers to students in each department, ordered by their marks.
Solution:

```
SELECT name, dept_name, marks,
    ROW_NUMBER() OVER (PARTITION BY dept_name ORDER BY marks DESC) AS
row_num
FROM student;
```

Comparison: RANK() vs DENSE_RANK() vs ROW_NUMBER()

Function	Description	Example Output
RANK()	Assigns ranks with gaps for ties	1, 2, 2, 4
DENSE_RANK()	Assigns ranks without gaps	1, 2, 2, 3
ROW_NUMBER()	Assigns a unique number to each row	1, 2, 3, 4

Introduction to Regular Expressions in MySQL

MySQL provides **pattern matching operations** through **`REGEXP`** (and its synonym **`RLIKE`**) to enhance search utilities. Regular expressions help match text data based on complex patterns, offering more precision than basic search methods.

- **REGEXP**: Operator for regular expression pattern matching.
- **RLIKE**: Synonym for REGEXP.
- **Case-Insensitive**: MySQL regular expressions are **not case-sensitive**.
- **Escape Character**: Use **double backslashes (`\\`)** to escape special characters.

Key Metacharacters in MySQL Regular Expressions

Pattern	Description
`*`	Zero or more instances of the preceding element
`+`	One or more instances of the preceding element
`.`	Matches any single character
`?`	Matches zero or one instance of the preceding element
`^`	Matches the **beginning** of a string
`$`	Matches the **end** of a string
`[abc]`	Matches any single character in the brackets
`[^abc]`	Matches any single character **not** in the brackets
`[A-Z]`	Matches any uppercase letter
`[a-z]`	Matches any lowercase letter
`[0-9]`	Matches any digit from 0 to 9
`[[:<:]]`	Matches the **beginning of words**
`[[:>:]]`	Matches the **end of words**
`[:class:]`	Matches a character class (e.g., `[:alpha:]`, `[:digit:]`)
`p1	p2
`{n}`	Exactly `n` occurrences of the preceding element
`{m,n}`	Between `m` and `n` occurrences of the preceding element

IFNULL VS COALESCE

When working with databases, dealing with missing or unknown data is a common challenge. In SQL, this is represented by NULL values. Handling NULL values correctly is crucial for ensuring your data operations are accurate and consistent.

Two key functions that help manage NULL values are COALESCE and IFNULL (or ISNULL, depending on the database system). While both functions aim to handle NULL values, they have different features and uses.

FEATURE	IFNULL()/ ISNULL()	COALESCE
Number of Arguments	2	Multiple
Standardization	Not part of SQL standard (specific to MySQL and SQL Server)	Part of SQL standard
Versatility	Limited for 2 arguments	We can handle multiple arguments
Return Type	Returns the data type of the first argument	Returns the data type of the first non-NULL argument

Performance	Possibly more effective in certain databases for two-argument scenarios	Effective in managing multiple possibilities NULL values
Portability	Less portable (database-specific)	More portable across different SQL databases

Company Related Questions and Platform

Question 1: LINK

Pivot the Occupation column in OCCUPATIONS so that each Name is sorted alphabetically and displayed underneath its corresponding Occupation. The output column headers should be Doctor, Professor, Singer, and Actor, respectively. Note: Print NULL when there are no more names corresponding to an occupation.

Question 2: LINK

You are given a table, Projects, containing three columns: Task_ID, Start_Date and End_Date. It is guaranteed that the difference between the End_Date and the Start_Date is equal to 1 day for each row in the table.

If the End_Date of the tasks are consecutive, then they are part of the same project. Samantha is interested in finding the total number of different projects completed. Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that has the same number of completion days, then order by the start date of the project.

Question 3: LINK

Write a solution to report the sum of all total investment values in 2016 tiv_2016, for all policyholders who: have the same tiv_2015 value as one or more other policyholders, and are not located in the same city as any other policyholder (i.e., the (lat, lon) attribute pairs must be unique).

Round tiv_2016 to two decimal places.

Question 4: LINK

You are the restaurant owner and you want to analyze a possible expansion (there will be at least one customer every day). Compute the moving average of how much the customer paid in a seven days window (i.e., current day + 6 days before). average_amount should be rounded to two decimal places.

Return the result table ordered by visited_on in ascending order.

Question 5: LINK

A company's executives are interested in seeing who earns the most money in each of the company's departments. A high earner in a department is an employee who has a salary in the top three unique salaries for that department. Write a solution to find the employees who are high earners in each of the departments.

Return the result table in any order.

Question 6: LINK

You are given three tables: Students, Friends and Packages.

Students contain two columns: ID and Name.

Friends contains two columns: ID and Friend_ID (ID of the ONLY best friend).

Packages contain two columns: ID and Salary (offered salary in $ thousands per month).

Write a query to output the names of those students whose best friends got offered a higher salary than them. Names must be ordered by the salary amount offered to the best friends. It is guaranteed that no two students get the same salary offer.

Question 7: LINK

You are given a table, Functions, containing two columns: X and Y.

Two pairs (X1, Y1) and (X2, Y2) are said to be symmetric pairs if X1 = Y2 and X2 = Y1.

Write a query to output all such symmetric pairs in ascending order by the value of X. List the rows such that X1 ≤ Y1.

Question 8: LINK

Write a solution to find the prices of all products on 2019-08-16. Assume the price of all products before any change is 10.

Return the result table in any order.

Question 9: LINK

There is a queue of people waiting to board a bus. However, the bus has a weight limit of 1000 kilograms, so there may be some people who cannot board. Write a solution to find the person_name of the last person that can fit on the bus without exceeding the weight limit. The test cases are generated such that the first person does not exceed the weight limit. Note that only one person can board the bus at any given turn.

Question 10: LINK

Write a solution to calculate the number of bank accounts for each salary category.

The salary categories are:

"Low Salary": All the salaries are strictly less than $20000.

"Average Salary": All the salaries in the inclusive range [$20000, $50000].

"High Salary": All the salaries strictly greater than $50000.

The result table must contain all three categories. If there are no accounts in a category, return 0.

Return the result table in any order.